

Assignment 4 Question 3

Nur Alya Maisarah Abd Karim, A00268252

Contents

1	Loading the Libraries and Import Dataset	2
2	Check Missing Values and Removal of Rows	2
3	Remove Duplicates and Correcting Data Types	3
4	Standardizing Categorical Variable	5
5	Attempts at Extracting Data from Description Column	6
6	Extract Vehicle Model Names	9
7	Remove Outliers	10
8	Visualization	12
9	Conclusion	18

1 Loading the Libraries and Import Dataset

To begin the data analysis, loading the necessary libraries and importing the dataset is a crucial first step. The readr library is used to efficiently read and handle CSV files, while other libraries such as dplyr for data manipulation, ggplot2 for visualization, lubridate for date handling, and stringr for string manipulation are also included to support the entire analysis process. These libraries are loaded with the library() function, making all their tools available for use. The dataset, “CarsDataset.csv,” is then imported into R using the read_csv() function from the readr package, which ensures that the data is read into a variable (e.g., cars_data) in a format ready for manipulation. This foundational step sets the stage for data cleaning, transformation, and in-depth analysis.

```
# Load the necessary library
library(readr)
library(ggplot2)
library(tinytex)
library(tidyr)
library(dplyr)
library(stringr)
library(lubridate)
library(stringi)

# Load the dataset into R
carsdataset <- read.csv("CarsDataset.csv")

# Manually set the first row as column names
colnames(carsdataset) <- carsdataset[1, ]
# Remove the first row after setting it as header
carsdataset <- carsdataset[-1, ]
```

2 Check Missing Values and Removal of Rows

To prepare the dataset for analysis, the first step is to check for missing values, as these can lead to errors or inaccurate results. Using summary(cars_data), we get a quick overview of each column, including any missing values. To dive deeper, colSums(is.na(cars_data)) counts how many values are missing in each column. This step helps identify incomplete rows that need to be addressed, ensuring we only work with reliable data.

Once the missing values are identified, rows with any missing entries are removed using the filter_all() function. This function creates a new dataset that contains only complete rows. Instead of using na.omit(), we use filter_all() to ensure that any rows with empty strings or rows where values are just spaces are also removed. After cleaning, it's important to confirm the results by checking the cleaned dataset for missing values again using colSums(is.na(cleaned_data)). Finally, dim(cleaned_data) is used to check the updated number of rows and columns, ensuring everything is consistent. These steps ensure the dataset is complete and ready for meaningful analysis without the risk of skewed results due to missing information.

Another important step is to remove rows with a price equal to 0. This is essential for ensuring that the dataset contains only valid and relevant data. A price of 0 is often a sign of missing or incorrect information, especially in the context of car listings, where a vehicle's price should always be greater than zero. Including rows with a price of 0 could introduce errors into the analysis, particularly when performing price-related calculations or visualizations. By filtering out these rows, the dataset is refined to include only realistic entries, ensuring that subsequent analysis is based on accurate and meaningful data. This step is crucial for maintaining the integrity of the analysis and preventing the inclusion of erroneous values that could compromise the overall findings.

```

# Convert all text columns to UTF-8 encoding to avoid the invalid string issue
carsdataset[] <- lapply(carsdataset, function(x) {
  if (is.character(x)) {
    return(iconv(x, from = "latin1", to = "UTF-8", sub = ""))
  } else {
    return(x)
  }
})

# Remove rows with any missing values, empty strings or strings with just spaces
clean_car <- carsdataset %>%
  filter_all(all_vars(!is.na(.) & . != "" & trimws(.) != ""))

# Remove rows where price is equal to 0
clean_car <- clean_car %>%
  filter(price != 0)

# Check for the counts of missing values (NA) in the cleaned dataset
missing_values <- colSums(is.na(clean_car))

# Check for blank values (empty strings) in the cleaned dataset
# Counts blank entries (empty strings) per column
blank_values <- colSums(clean_car == "")

# Combine both missing and blank values check
total_missing <- missing_values + blank_values

# Display the results
print(total_missing)
dim(clean_car)

```

3 Remove Duplicates and Correcting Data Types

To prepare the dataset for meaningful analysis, it is important to address common data quality issues such as duplicates, incorrect data types, and inconsistent values. Duplicates in the dataset can lead to inflated counts, inaccurate averages, and misleading conclusions. The `distinct()` function from the `dplyr` package is used to remove duplicate entries, ensuring each observation is unique and the analysis reflects accurate, independent records.

Another critical step in data cleaning is ensuring that all variables have the correct data types. For instance, columns like `price`, `odometer`, and `id` may be incorrectly imported as character data types, preventing proper mathematical operations such as summing or averaging. These columns are converted to numeric using the `as.numeric()` function, making them ready for analysis and statistical computations. Similarly, the `posting_date` column, initially stored as a string, is transformed into a `Date` format using the `ymd_hms()` function from the `lubridate` package. This ensures that time-based analyses, such as filtering by date or grouping by year, are accurate and functional.

The `cylinders` column, which may contain non-numeric characters, is also cleaned using the `gsub()` function to remove any non-numeric symbols, ensuring that only relevant numeric values are retained. In cases where the `cylinders` column contains empty values or invalid data, they are replaced with a default value such as “others” to maintain consistency. Additionally, empty or invalid numeric values in columns like `price` or `odometer` are replaced with 0 to prevent errors in calculations and ensure the dataset remains complete.

After cleaning the data types and correcting invalid values, functions like `summary()` and `str()` are used to check the structure of the dataset for any remaining issues such as missing values or data type mismatches.

By addressing these common data cleaning steps-removing duplicates, correcting data types, handling inconsistencies, and replacing invalid entries with default values-the dataset is refined and ready for accurate and meaningful analysis. These improvements ensure that the dataset is reliable, reducing the likelihood of errors in downstream analysis and providing a solid foundation for extracting valuable insights.

```
#Convert 'price', 'year', 'odometer' to numeric
clean_car$price <- as.numeric(clean_car$price)
clean_car$odometer <- as.numeric(clean_car$odometer)
clean_car$id <- as.numeric(clean_car$id)

#Convert 'posting_date' to Date format
clean_car$posting_date <- ymd_hms(clean_car$posting_date, tz = "UTC")

#Convert 'posting_date' to Date format and keep only the date
clean_car$posting_date <- as.Date(ymd_hms(clean_car$posting_date))

#Keep the structure of the dataset intact
clean_car$cylinders <- gsub("[^0-9]", "", clean_car$cylinders)

#Handle rows with no numeric values
clean_car$cylinders[clean_car$cylinders == ""] <- "others"

#Verify the structure of the cleaned data
str(clean_car)

#Now remove duplicates
clean_car <- distinct(clean_car)

#Check for any remaining issues
summary(clean_car)
```

4 Standardizing Categorical Variable

Standardizing categorical variables and cleaning text fields is a vital step in the data preparation process, as it helps eliminate inconsistencies and ensures uniformity across the dataset. Categorical variables such as `region`, `fuel`, `title_status`, and `condition` are often inconsistent due to variations in capitalization, spacing, and formatting. To address this, the `tolower()` and `toupper()` functions from the `dplyr` package are used to convert the values in these columns to lowercase (for `region`, `fuel`, `title_status`, and `condition`) and uppercase (for `state`). These functions help eliminate issues caused by inconsistent capitalization, which can interfere with accurate analysis, grouping, or aggregation of data. Additionally, the `trimws()` function is used to remove any leading or trailing whitespace, ensuring that the data is consistently formatted. These steps help to create a uniform dataset that can be easily grouped and compared, improving its overall reliability for further analysis.

For text fields such as `description` and `manufacturer`, the cleaning process focuses on eliminating unwanted characters, ensuring the data is both readable and suitable for analysis. The `iconv()` function is first applied to convert the text into UTF-8 encoding, which ensures consistent character representation and prevents misinterpretation of special characters due to different encodings. After that, the `gsub()` function is used to remove non-alphanumeric characters, except for essential symbols like the dollar sign (in the case of `description`). This function strips out any special characters that may have been inadvertently included in the dataset, which can interfere with processing or cause errors during analysis. Furthermore, additional steps are taken to remove non-ASCII characters, such as special symbols or characters from different alphabets, that may have been mistakenly included during data entry or import. Specific unwanted characters, such as those resulting from encoding issues (e.g., “ÃÂ”), are also removed using targeted `gsub()` commands. Finally, the `trimws()` function is applied again to remove any excess whitespace from the text fields, ensuring they are clean and free from irrelevant characters.

By standardizing the categorical variables and cleaning the text fields, the dataset becomes more consistent and easier to work with. These steps ensure that all text is in a uniform format, free from errors, and ready for further analysis or modeling. This level of cleanliness and consistency is crucial for generating accurate insights and making reliable conclusions from the data, whether the goal is to conduct exploratory analysis, develop predictive models, or generate business insights.

```
#Region to lowercase
clean_car$region <- tolower(trimws(clean_car$region))

#Fuel to lowercase
clean_car$fuel <- tolower(trimws(clean_car$fuel))

#Status to lowercase
clean_car$title_status <- tolower(trimws(clean_car$title_status))

#Status to lowercase
clean_car$condition <- tolower(trimws(clean_car$condition))

#Standardize state to uppercase
clean_car$state <- toupper(trimws(clean_car$state))

#Clean text field and remove special characters
# Ensure descriptions are in UTF-8 encoding
clean_car$description <-
  iconv(clean_car$description, from = "latin1", to = "UTF-8")

#Remove special characters
clean_car$description <- gsub("[^[:alnum:][:space:]]$",
  "", clean_car$description)
```

```

#Remove leading/trailing whitespace
clean_car$description <- trimws(clean_car$description)

#Remove non-ASCII characters (if any)
clean_car$description <-
  gsub("[^[:print:]]", "", clean_car$description)

#Remove specific unwanted characters
clean_car$description <-
  gsub("[ÃÄ]", "", clean_car$description)

#Clean the 'manufacturer' column
clean_car$manufacturer <-
  gsub("[^[:alnum:][:space:]]", "", clean_car$manufacturer)
clean_car$manufacturer <-
  trimws(clean_car$manufacturer)

#Check the cleaned columns
head(clean_car$region)
head(clean_car$state)
head(clean_car$description)
head(clean_car$title)
head(clean_car$manufacturer)

```

5 Attempts at Extracting Data from Description Column

Efforts to extract and clean vehicle prices and models from descriptions using regular expressions (regex) and custom functions faced several challenges, leading to incomplete and unreliable results. The primary goal was to isolate dollar amounts, such as “\$14,999,” while filtering out promotional text like “down payment,” “finance incentives,” or “MUST SELL.” Although regular expressions were employed to identify price patterns, significant issues arose, particularly with truncating prices (e.g., “\$14,999” became “149”), resulting in data loss and inaccuracies. This truncation undermined the usability of the extracted prices for meaningful analysis.

One primary reason for this issue was the improper handling of special characters, such as commas and dollar signs, which are common in numeric representations of prices. In some cases, cleaning steps may have inadvertently stripped out commas and trailing digits instead of preserving the entire numeric string. Another potential cause was inconsistencies within the descriptions themselves, including multiple dollar amounts, which led to incorrect regex matches.

The unstructured nature of the data added another layer of complexity. Descriptions often included inconsistent formatting, embedded promotional language, and varied ways in which prices and models were presented. This variability made it difficult for regular expressions to reliably capture the intended information without inadvertently excluding or misinterpreting parts of the text.

Furthermore, extracting vehicle models from the descriptions proved equally challenging. Descriptions often contained multiple details, such as the year, price, and promotional offers, with no standardized structure. This lack of uniformity hindered the accurate isolation of model names, even after applying cleaning and extraction techniques. Ambiguities, such as missing or partial information, further complicated the process.

These challenges underline the limitations of using regular expressions alone for extracting and cleaning data from highly variable and noisy unstructured text. While regex can be a powerful tool for identifying specific patterns, it struggles when dealing with complex, inconsistent, and unpredictable

data. In this context, the extraction of prices involved using the following regular expression pattern: `\\$\\s?\\d{1,3}(?:,\\d{3})*(?:\\.\\d{2})?`. This pattern was designed to match price values like "1,999.99" by capturing the dollar sign (optional space (`\\s?`), and the numeric part of the price (with optional commas and decimals). Despite these efforts, issues such as truncation and incorrect matches arose due to irregularities in the descriptions.

To address the issues of truncation and cleanup, a series of custom functions were applied to process the descriptions, extract the prices, and clean them for analysis. The `clean_description` function, using the `gsub()` function from base R, was designed to remove promotional or finance-related text from the descriptions. It replaced unwanted phrases like "down payment" and "MUST SELL" with an empty string, ensuring these did not interfere with price extraction.

The `extract_prices` function utilized the `str_extract_all()` function from the `stringr` package to extract price values from cleaned descriptions. The regular expression pattern was applied using `str_extract_all()`, which returned all matches of price patterns in the text. This approach handled cases where multiple prices appeared in a single description and ensured that only the numeric portion was retained by removing dollar signs, commas, etc., using the `gsub()` function.

Once the prices were extracted, the `clean_prices` function was used to process the extracted prices and convert them into a numeric format. This function applied `as.numeric()` to the cleaned prices, ensuring that non-numeric characters were removed, and the values were ready for analysis.

The cleaned dataset, `clean_cartest`, included columns for the original descriptions, the cleaned descriptions, the extracted prices, and the cleaned prices. These steps ensured that the data was prepared for further analysis, with cleaned price information available for meaningful insights.

While regular expressions proved useful for identifying price patterns, their limitations were evident when dealing with unstructured, inconsistent, and noisy data. The truncation of prices and inability to accurately extract models were significant challenges. These issues emphasize the need for more advanced methods, such as natural language processing (NLP) or machine learning, to handle the complexity of real-world unstructured text data effectively. The packages used in the cleaning process included `stringr` for string manipulation (specifically `str_extract_all()`), and base R functions like `gsub()` and `as.numeric()` for regular expression matching and numeric conversion.

```
clean_cartest<- clean_car

#Function to clean the description
clean_description <- function(description) {
  if (is.na(description) || !is.character(description)) return(NA)

  #Remove promotional/finance-related text
  cleaned_description <- tryCatch({
    gsub("\\$\\d+\\s?off\\s?finance|down
        \\s?payment|finance\\s?incentive|MUST\\sSELL",
        "", description, ignore.case = TRUE)
  }, error = function(e) {
    return(description)
  })

  return(cleaned_description)
}

#Refined function to capture prices correctly
extract_prices <- function(description) {
  if (is.na(description)) return(NA)
```

```

#Regex pattern to match all prices
#Matches prices like $1,999.99
price_pattern <- "\\$\\s?\\d{1,3}(?:,\\d{3})*(?:\\.\\d{2})?"

#Extract all prices based on the pattern
matches <- str_extract_all(description, price_pattern)[[1]]

if (length(matches) == 0) return(NA) # Return NA if no price found

#Clean and process each extracted price
cleaned_prices <- sapply(matches, function(price) {
  #Remove the dollar sign and commas, leave the full number intact
  price_cleaned <- gsub("[^0-9.]", "", price)
  return(price_cleaned)
})

# Return the cleaned list of prices as a character vector
return(cleaned_prices)
}

#Convert extracted prices into numeric format
clean_prices <- function(prices) {
  if (is.null(prices) || length(prices) == 0) return(NA)

  #Process each price in the vector
  numeric_prices <- sapply(prices, function(price) {
    if (is.na(price)) return(NA)

    #Convert each price element to numeric
    #Removes unwanted characters like dollar signs, commas
    price_cleaned <- as.numeric(price)

    return(price_cleaned)
  })

  # Return the numeric values
  return(numeric_prices)
}

#Assuming clean_car is your dataset with descriptions
clean_cartest <- clean_car # Your original dataset

#Clean descriptions by removing unwanted text
clean_cartest$cleaned_description <-
  sapply(clean_cartest$description, clean_description)

#Extract prices using the refined function (will capture all prices)
clean_cartest$extracted_price <-
  sapply(clean_cartest$cleaned_description, extract_prices)

#Clean the extracted prices (removes non-numeric characters)
clean_cartest$cleaned_price <-
  sapply(clean_cartest$extracted_price, clean_prices)

```



```
#View the result
head(clean_cartest
      [, c("description", "cleaned_description",
            "extracted_price", "cleaned_price")])
```

6 Extract Vehicle Model Names

This code focuses on an attempt to extract vehicle model names from the `description` column of the dataset and clean the data to ensure its suitability for analysis, using the `dplyr` and `stringr` packages from R.

Initially, a new column, `model_extracted`, is created using `mutate()` from the `dplyr` package, combined with the `str_extract()` function from the `stringr` package. The `str_extract()` function is applied to identify specific patterns in the `description` field, such as the text that appears after the keyword “Type:” or a four-digit year. The regular expression used in the first `mutate()` function targets the text that follows these patterns, effectively isolating the vehicle model names.

For entries containing the promotional phrase “See more vans at valuecargovans.com,” the script refines the extraction process further. In these cases, the `model_extracted` field is updated to capture the model name following the four-digit year, ensuring more accurate identification of the model in descriptions with such promotional content. This second modification to the `model_extracted` column uses `ifelse()` from `dplyr`, combined with `str_detect()` from `stringr`, to check for this specific phrase and apply a more suitable extraction pattern.

After the model names are extracted, the dataset undergoes an additional cleaning step. Using `filter()` from `dplyr`, rows containing promotional or irrelevant keywords such as “CREDIT AMNESTY,” “SE HABLA ESPANOL,” “WORLDAUTOSINC,” “FREE Transportation,” “PAYMENTS AS LOW AS,” and “down payment” are removed from the dataset. These keywords are considered noise, as they do not contribute to the core analysis of vehicle models and may obscure valuable insights. The `grepl()` function is used to identify these phrases in the `description` column and exclude rows where they appear, ensuring that only relevant records are retained.

The final cleaned dataset, stored as `cleaned_carsdataset`, now contains only the relevant rows, with unnecessary promotional content and irrelevant text filtered out. The extracted model names are now more accurate, and the dataset is more focused and structured, ensuring better quality for subsequent analysis. By cleaning the dataset and refining the extraction methods, the dataset is optimized for analysis, with a more precise and reliable `model_extracted` column for further exploration. The use of the `dplyr` and `stringr` packages ensures efficient and effective data manipulation and text extraction throughout the process.

```
library(dplyr)
library(stringr)
#Extract model names from descriptions
carsdataset <- carsdataset %>%
  mutate(
    model_extracted = str_extract(description,
                                   "(?<=Type: )([\\w\\s-]+
                                   (?=\\s*\\b))")
  )

#Replace rows
carsdataset <- carsdataset %>%
  mutate(
    model_extracted = ifelse(
      str_detect(description, "See more vans at valuecargovans.com"),
```

```

    str_extract(description, "(?<=\\d{4}\\s)([\\w\\s-]+(?:\\s*\\b))"),
    model_extracted
  )
)

# Extract model names from the description
carsdataset <- carsdataset %>%
  mutate(
    model_extracted = str_extract(description,
                                   "(?<=Type:|\\d{4}\\s)([\\w\\s-]+(?:\\s*\\b))")
  )

# Combine keywords into a single line
cleaned_carsdataset <- carsdataset %>%
  filter(!grepl("CREDIT AMNESTY|SE HABLA ESPANOL|WORLDAUTOSINC|FREE Transportation|PAYMENTS AS LOW AS|d

# View a sample of the cleaned dataset
head(cleaned_carsdataset)

```

7 Remove Outliers

This code is designed to identify and remove outliers in the `price` column of the `clean_car` dataset, using the interquartile range (IQR) method to detect extreme values. It also ensures that the dataset is free from unrealistic or invalid prices before proceeding with the analysis.

The first step in the process involves removing rows with unrealistic or invalid price values. Specifically, any records with a `price` less than or equal to zero are excluded using the `filter()` function from the `dplyr` package, ensuring that only valid prices remain in the dataset.

Next, a custom function, `IQR_outliers()`, is defined to detect outliers using the IQR method. This function first calculates the 25th and 75th percentiles (quartiles) of the `price` column using the `quantile()` function, with missing values excluded (`na.rm = TRUE`). The IQR is then computed by subtracting the 25th percentile from the 75th percentile. Using this IQR, the lower and upper bounds for potential outliers are calculated, with any `price` values falling below the lower bound or above the upper bound considered outliers. The function returns these bounds as a vector.

The outlier bounds are then applied to the `price` column of the dataset. The `price` values that fall outside of these bounds are identified using a logical condition, and the rows containing these outliers are removed from the dataset, resulting in `clean_car_no_outliers`. This cleaned dataset now excludes any extreme price values that could distort the analysis.

To visualize the impact of the outlier removal, a boxplot is generated using the `boxplot()` function, displaying the distribution of `price` values after the outliers have been removed. The boxplot helps to clearly illustrate the spread of the data and highlight any remaining values that may still be considered outliers.

By eliminating outliers, the dataset becomes more suitable for analysis, as extreme values can significantly skew results. The cleaned dataset, `clean_car_no_outliers`, is now ready for further analysis with reduced risk of distortions caused by these unusual price points. The use of the `dplyr` package ensures efficient filtering, and the outlier detection and visualization are facilitated by base R functions like `quantile()` and `boxplot()`.

```

#Remove unrealistic or invalid price values
clean_car <- clean_car %>%
  filter(price > 0)

```

```

#Define a function to detect outliers using the IQR method
IQR_outliers <- function(column) {
  quan <- quantile(column, na.rm=TRUE) # Calculate quartiles
  IQR = as.integer(quan["75%"] - quan["25%"])

  #Calculate the lower and upper bounds for outliers
  lower_bound = quan["25%"] - 1.5 * IQR
  upper_bound = quan["75%"] + 1.5 * IQR

  return(c(lower_bound, upper_bound)) # Return bounds
}

#Apply the function to the 'price' column in clean_car dataset
outlier_bounds <- IQR_outliers(clean_car$price)

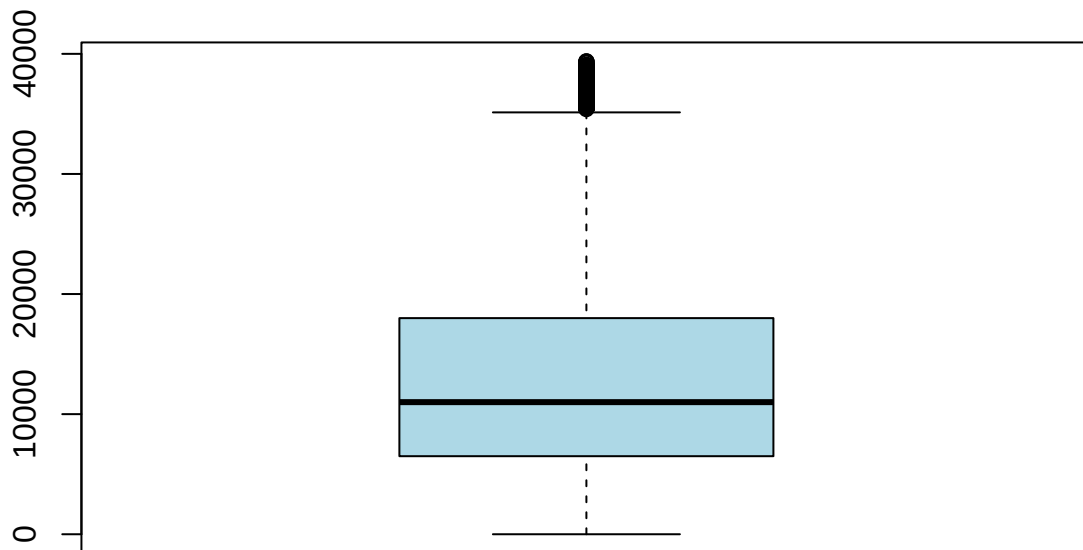
#Identify rows with outliers in the 'price' column
outliers <- clean_car$price <
  outlier_bounds[1] | clean_car$price > outlier_bounds[2]

#Remove outliers from the dataset
clean_car_no_outliers <- clean_car[!outliers, ]

#Visualize the distribution of prices with a boxplot
boxplot(clean_car_no_outliers$price, main =
  "Price Boxplot (After Removing Outliers)",
  col = "lightblue")

```

Price Boxplot (After Removing Outliers)



```
#Check the cleaned dataset  
head(clean_car_no_outliers)  
  
#View the cleaned data  
head(clean_car_no_outliers)
```

8 Visualization

This code is aimed at visualizing the cleaned dataset to verify its accuracy and integrity before performing any further analysis. Visualization is a powerful technique for uncovering patterns, trends, and potential issues such as outliers or skewed data that may not be immediately obvious from raw data alone. By utilizing various types of plots, such as bar charts, pie charts, and boxplots, the data can be examined from different perspectives, helping to ensure that it is ready for analysis without introducing any biases or errors.

The first part of the visualization process involves grouping the `clean_car_no_outliers` dataset by state and counting the number of vehicles in each state. A bar plot is then created to show the distribution of cars across states. The `ggplot2` package is used to generate this plot, with customization options to adjust the width of the bars and format the y-axis labels for readability. This bar plot helps to identify any imbalances in the number of cars available in different states.

Next, the code calculates the average price for the top three manufacturers (`ford`, `chevrolet`, and `toyota`). A bar chart is created to display the number of cars available by manufacturer, allowing for an understanding of the dataset's composition in terms of car brands. The chart is formatted with the `ggplot2` package, which is customized by flipping the axes for better readability and styling the text.

A pie chart is also generated to visualize the distribution of car fuel types. The dataset is grouped by `fuel`, and the number of occurrences for each fuel type is counted. The `ggplot2` package is used to create the pie chart, which is further customized by defining custom colors using the `scale_fill_manual()` function. This pie chart provides insights into the proportions of different fuel types within the dataset, helping to identify any trends or imbalances.

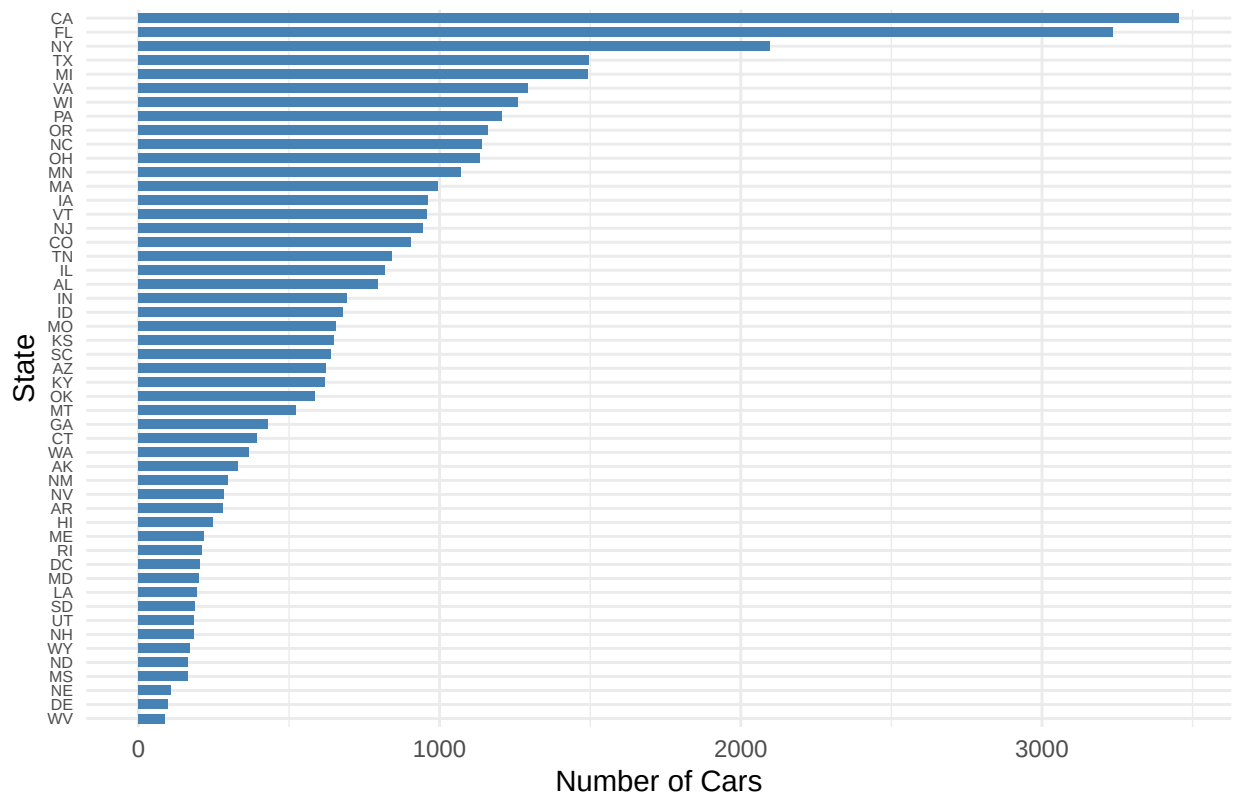
Finally, the code aggregates the average car price by both `condition` and `title_status`. A bar plot is created to visualize how the average price varies depending on the car's condition and title status. The `RColorBrewer` package is used to define a color palette based on the unique `title_status` levels. The plot is customized with various elements like rotating x-axis labels and adjusting the theme for a cleaner appearance. This visualization provides a detailed comparison of car prices based on their condition and title status, allowing for a better understanding of the dataset's characteristics.

These visualizations, created using the `ggplot2` and `RColorBrewer` packages, are key to ensuring that the data is clean, well-structured, and free from any anomalies. They also serve as a valuable tool for understanding the distribution of key variables such as car prices, manufacturers, and fuel types, which will aid in the analysis and interpretation of the dataset.

```
# Group by state and count the number of vehicles per state
by_state <- clean_car_no_outliers %>%
  group_by(state) %>%
  count() %>%
  arrange(n)

# Create a bar plot with customized y-axis text
ggplot(by_state, aes(x = n, y = reorder(state, n))) +
  geom_col(fill = "steelblue", width = 0.7) + # Adjust width for gaps
  labs(
    title = "Used Cars by Each State",
    x = "Number of Cars",
    y = "State"
  ) +
  theme_minimal() +
  theme(
    axis.text.y = element_text(size = 5.5) # Customize y-axis text
  )
```

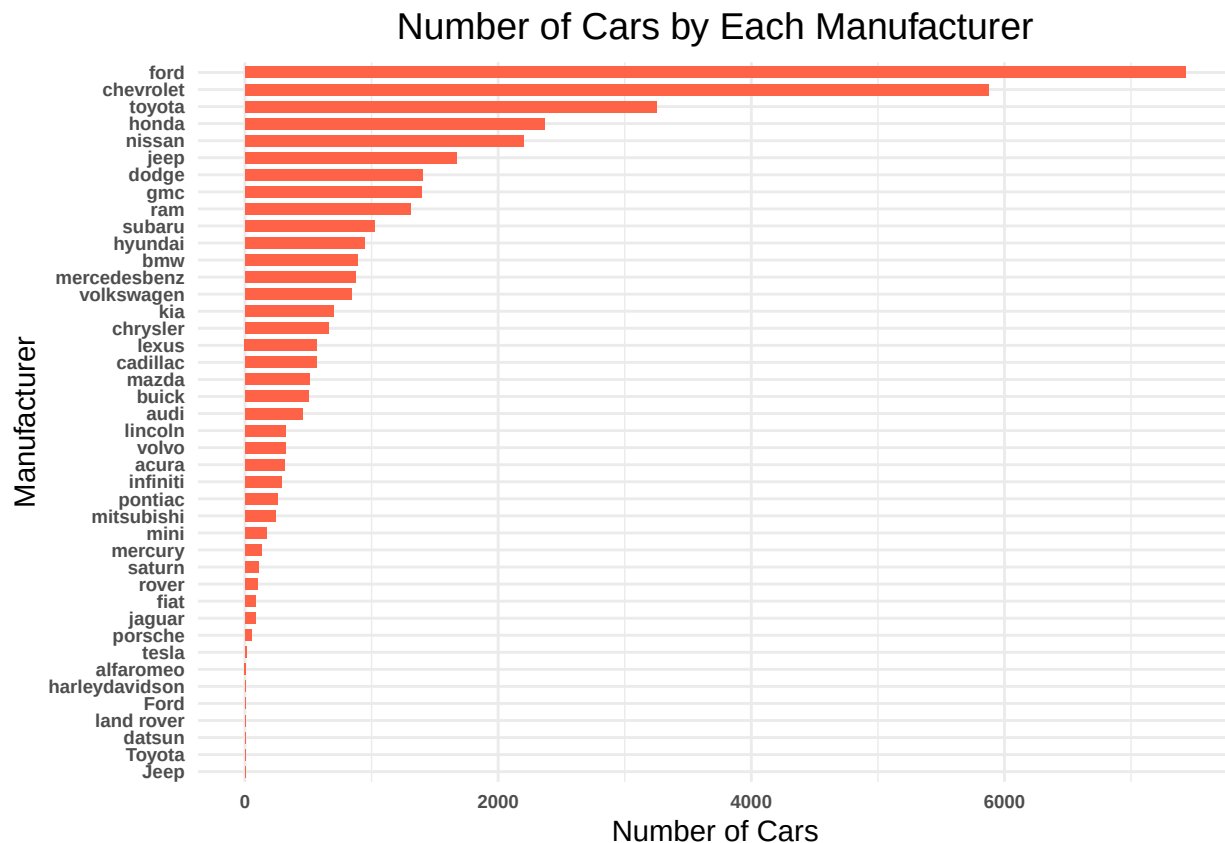
Used Cars by Each State



```
# Calculate the average price for the top 3 manufacturers
avg_prices <- clean_car_no_outliers %>%
  filter(manufacturer %in% c("ford", "chevrolet", "toyota")) %>%
  group_by(manufacturer) %>%
  summarise(avg_price = round(mean(price, na.rm = TRUE), 2))

# Create a bar chart for the count of vehicles by manufacturer
by_manf <- clean_car_no_outliers %>%
  group_by(manufacturer) %>%
  count() %>%
  arrange(desc(n))

ggplot(by_manf, aes(x = reorder(manufacturer, n), y = n)) +
  geom_col(fill = "tomato", width = 0.7) + # Adjust width for spacing
  coord_flip() + # Flip the axes for better readability
  labs(
    title = "Number of Cars by Each Manufacturer",
    x = "Manufacturer",
    y = "Number of Cars"
  ) +
  theme_minimal() +
  theme(
    axis.text = element_text(size = 7, face = "bold"),
    plot.title = element_text(hjust = 0.5, size = 14)
  )
```

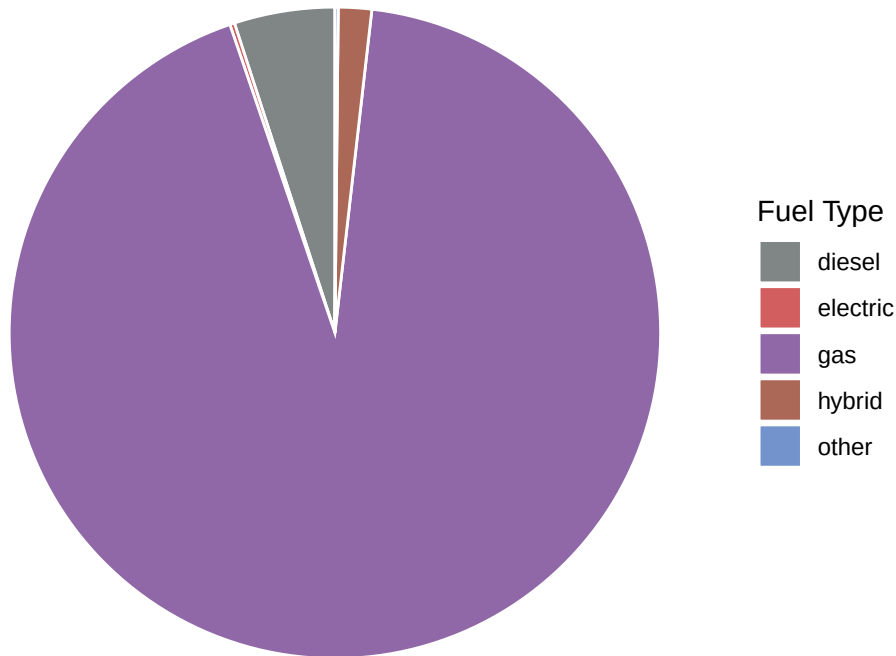


```
# Group by fuel type and count occurrences
by_fuel <- clean_car_no_outliers %>%
  group_by(fuel) %>%
  count() %>%
  drop_na()

# Define valid hex colors
colors <- c('#808585', '#D35E60', '#9067A7', '#AB6857', '#7293CB')

# Plot a pie chart using ggplot2
ggplot(by_fuel, aes(x = "", y = n, fill = fuel)) +
  geom_bar(stat = "identity", width = 1, color = "white") +
  coord_polar("y", start = 0) +
  labs(title = "Car Fuel Type", fill = "Fuel Type") +
  scale_fill_manual(values = colors) +
  theme_void() + # Removes unnecessary axes and gridlines
theme(
  plot.title = element_text(hjust = 0.5, size = 14),
  legend.position = "right"
)
```

Car Fuel Type



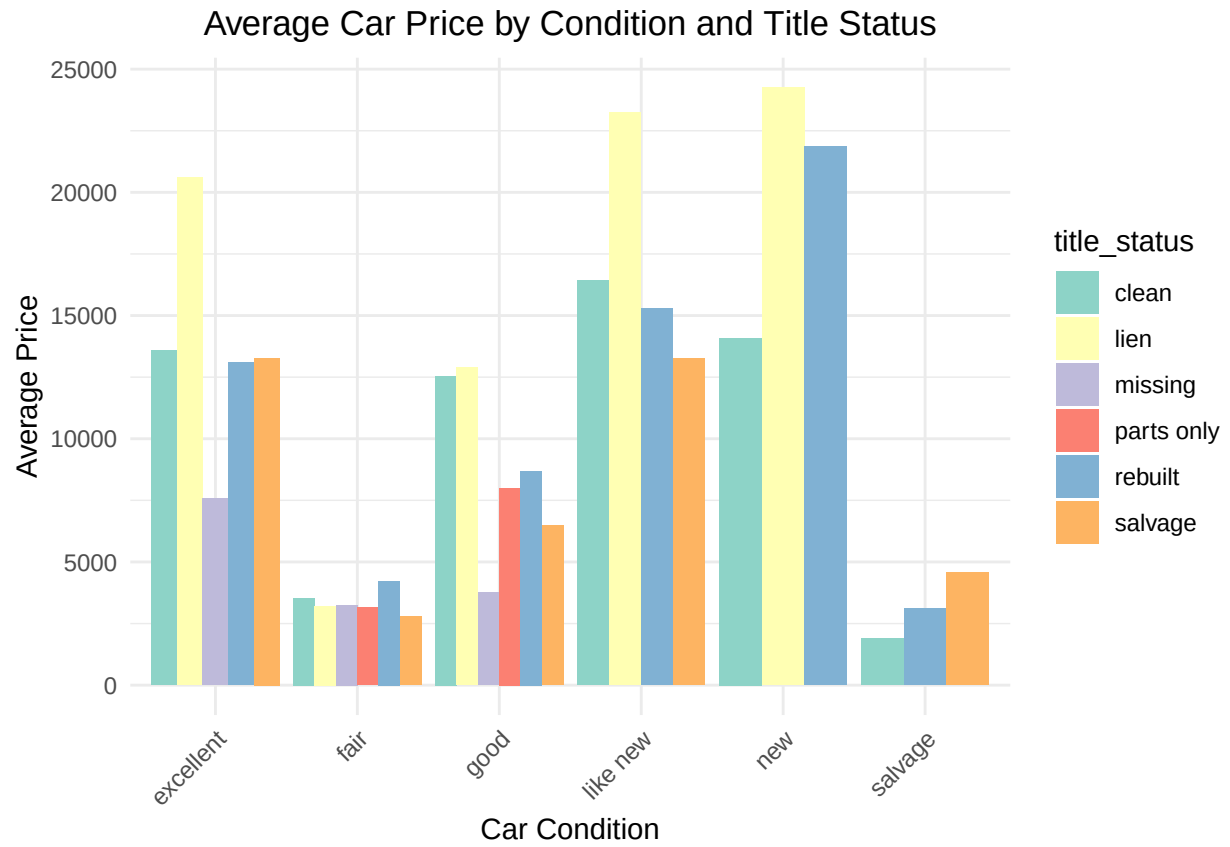
```
# Aggregate average car price by condition and title status
agg_data <- aggregate(price ~ condition + title_status, data = clean_car_no_outliers, FUN = mean, na.rm = TRUE)

# Check if the data has been correctly aggregated
print(agg_data)

# Get unique levels of 'title_status' to define a color palette
unique_title_status <- unique(agg_data$title_status)

# Define a color palette based on the number of unique levels
colors <- RColorBrewer::brewer.pal(length(unique_title_status), "Set3")

# Plot the aggregated data using ggplot2
ggplot(agg_data, aes(x = condition, y = price, fill = title_status)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(
    title = "Average Car Price by Condition and Title Status",
    x = "Car Condition",
    y = "Average Price"
  ) +
  scale_fill_manual(values = colors) + # Automatically assigns the colors
  theme_minimal() + # A clean theme
  theme(
    plot.title = element_text(hjust = 0.5), # Center title
    axis.text.x = element_text(angle = 45, hjust = 1) # Rotate x-axis labels for readability
  )
```

9 Conclusion

In conclusion, cleaning and preparing data for analysis is a crucial step to ensure the accuracy and reliability of the results. By handling missing values, removing duplicates, and correcting data types, we create a dataset that is consistent and ready for meaningful analysis. The process of identifying and removing outliers further refines the data, preventing extreme values from skewing the results. Additionally, standardizing categorical variables and cleaning text fields ensures uniformity and removes any inconsistencies in the dataset, ultimately improving the quality of the analysis.

Visualization plays a key role in testing and validating the cleaned data, providing a clear view of trends and patterns. By using techniques like boxplots and bar charts, we can quickly assess the distribution of key variables such as price and condition, identify any remaining anomalies, and confirm the effectiveness of the cleaning process. These visual checks help to ensure that the dataset is not only accurate but also ready for deeper analysis or modeling, ultimately leading to more reliable and insightful conclusions.